Lab 7 Setup and Run Guide: Introduction to ECS

lab07-ecs-intro.yaml builds the standing pieces of Lab 7 in one CloudFormation stack: the cluster, the task execution role, the log group, the Fargate task definition and the security group. You then start the task yourself with run-task, check that the application really started, and call it from your own machine. That hands-on part is the point of the lab, so it stays manual.

What the stack creates

README step	Resource
1	ECS cluster leap-mission-cluster-teamN
2	Task execution role leap-teamN-ecs-execution-role, trusting ecs-tasks.amazonaws.com, with the AWS-managed AmazonECSTaskExecutionRolePolicy
3	CloudWatch log group /leap/mission-service/teamN, and a Fargate task definition leap-mission-service-teamN for your Module 6 image, logging to that group
4	The task definition states the CPU architecture explicitly (RuntimePlatform), so you decide it on purpose
5	Security group leap-teamN-app-sg allowing the container port from 0.0.0.0/0. Wide open on purpose, for testing only

Steps 6 to 8 (run the task, read its logs, call its public IP) are done by you below. Names include your team number so four teams can share a region.

IMAGE URI: 760144414473.dkr.ecr.us-east-1.amazonaws.com/leap-mission-service:latest

ECR: leap-mission-service

ENV: Setup from module 3 (team-x.env)

Before you start

- Your team number (1 to 4) and region. The examples use us-east-1.
- If your application needs environment variables, a database or secrets to start, the template does not know about them. Add them to the team-x.env file.
- Run ‘source team-x.env’ to apply the environment variables.

Part 1: cluster, role, task definition

Step 1. Find your values (should be defined in team-X.env)

```
# export REGION=us-east-1

# export TEAM=X
```

Your repository and image tags

```
aws ecr describe-images --region $REGION --repository-name leap-mission-service \

  --query "imageDetails[].[imageTags[0],imagePushedAt]" --output table
```

export IMAGE= 760144414473.dkr.ecr.us-east-1.amazonaws.com/leap-mission-service:latest

The VPC

```
export VPC=$(aws ec2 describe-vpcs --region $REGION --filters "Name=tag:Name,Values=leap-team$TEAM-vpc" --query "Vpcs[0].VpcId" --output text)

echo $VPC
```

Note: update your team-X.env with this value (export VPC=<vpc id>

Step 2. Check the image architecture before running anything (README step 4)

```
aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin ${IMAGE%%/*}

docker manifest inspect $IMAGE | grep -i architecture
```

amd64 means X86_64. arm64 means ARM64 (common on Apple Silicon). If the output lists both, the image is multi-architecture and either works. Whatever you find, pass it in the next step. Also open the Dockerfile and read the EXPOSE line: that is the port, not 8080 by assumption.

Step 3. Deploy the stack

In CloudShell, use Actions, then Upload file, and choose lab07-ecs-intro.yaml. Then:

```
export ARCH=X86_64              # or ARM64, from step 2
export PORT=8080                # from the Dockerfile EXPOSE line
aws cloudformation deploy --region $REGION \
  --template-file lab07-ecs-intro.yaml \
  --stack-name lab7-team$TEAM \
  --capabilities CAPABILITY_NAMED_IAM \
  --tags team=$TEAM \
  --parameter-overrides Team=$TEAM VpcId=$VPC ImageUri=$IMAGE \
    CpuArchitecture=$ARCH ContainerPort=$PORT
```

Check Your Security Group and Export:

```
export SG=$(aws cloudformation describe-stacks --region $REGION --stack-name lab7-team$TEAM --query
"Stacks[0].Outputs[?OutputKey=='AppSecurityGroupId'].OutputValue" --output text)
echo "Using security group $SG”
```

Note: Update your team-X.env with this security group

Step 4. Read the outputs

```
aws cloudformation describe-stacks --region $REGION --stack-name lab7-team$TEAM \
  --query "Stacks[0].Outputs" --output table
```

Part 2: run it and prove it

Step 5. Pick a public subnet

```
aws ec2 describe-subnets --region $REGION --filters Name=vpc-id,Values=$VPC \
  --query "Subnets[].[SubnetId,AvailabilityZone,MapPublicIpOnLaunch,Tags[?Key=='Name']|[0].Value]" --
output table

export SUBNET=<a subnet from the public tier>
```

Use a public subnet, meaning its route table sends 0.0.0.0/0 to an internet gateway. A task in a subnet without that route cannot reach the internet to pull its image.

Step 6. Run the task

```
export CLUSTER=leap-mission-cluster-team$TEAM

TASK=$(aws ecs run-task --region $REGION --cluster $CLUSTER --launch-type FARGATE \
  --task-definition leap-mission-service-team$TEAM \
  --network-configuration
"awsvpcConfiguration={subnets=[$SUBNET],securityGroups=[$SG],assignPublicIp=ENABLED}" \
  --query "tasks[0].taskArn" --output text)

echo $TASK
```

Keep each command on one line, or make sure nothing follows the backslash when you split it.

Step 7. Wait for RUNNING

```
aws ecs wait tasks-running --region $REGION --cluster $CLUSTER --tasks $TASK

aws ecs describe-tasks --region $REGION --cluster $CLUSTER --tasks $TASK \
  --query "tasks[0].[lastStatus,stoppedReason]" --output text
```

If the wait fails, the task stopped. The stoppedReason line says why. See the troubleshooting table.

Step 8. Read the real logs

```
aws logs tail /leap/mission-service/team$TEAM --region $REGION --since 15m
```

RUNNING only means the container process started. The logs show whether the application really started. Look for your framework's startup banner and a line saying it is listening on your port. A stack trace or a missing setting means it did not.

Step 9. Call it from your own machine

```
ENI=$(aws ecs describe-tasks --region $REGION --cluster $CLUSTER --tasks $TASK \
  --query "tasks[0].attachments[0].details[?name=='networkInterfaceId'].value" --output text)
IP=$(aws ec2 describe-network-interfaces --region $REGION --network-interface-ids $ENI \
  --query "NetworkInterfaces[0].Association.PublicIp" --output text)
echo $IP
```

Then, from a terminal on your laptop (not CloudShell):

```
curl -i http://<IP>:8080/actuator/health
```

Use your own port and a path your application serves. Any real HTTP response counts, including a 401 or 403, because it proves the request reached the application. A connection timeout means the security group, the port, or the architecture is still wrong.

Verify

- The task reached RUNNING.
- The logs show a real application startup, not just a started container.
- A request from your own machine got an HTTP response.
- The execution role trusts only ecs-tasks.amazonaws.com and has the AmazonECSTaskExecutionRolePolicy attached:

```
aws iam list-attached-role-policies --role-name leap-team$TEAM-ecs-execution-role
```

Troubleshooting

Symptom	Likely cause and fix
stoppedReason: exec format error, or logs show it	Image and task architecture differ. Redeploy the stack with the other CpuArchitecture value, or rebuild the image with --platform linux/amd64.
CannotPullContainerError	The task could not pull the image. Check the image URI and tag, that the subnet is public, and that assignPublicIp is ENABLED.
Task goes STOPPED right after starting	The application exited. Read the logs. A missing environment variable, database or secret is the usual cause.
Out of memory, task killed	Raise Memory (and Cpu) and redeploy the stack, then run a new task. The memory value must be valid for the CPU value.
curl times out	Wrong port, or the security group does not allow it. Compare the stack's ContainerPort with the Dockerfile EXPOSE line. Also check the task really got a public IP.
Stack create fails on the log group or role	A resource with that name exists from an earlier attempt. Delete the old one or the old stack.
No output from aws logs tail	The container never started, or logging failed. Check the task's stopped reason and that the log group name matches the stack output.

Cleanup

Stop any running task. Leave the stack in place, because Module 8 builds directly on it:

```
aws ecs list-tasks --region $REGION --cluster $CLUSTER --query "taskArns[]" --output text

aws ecs stop-task --region $REGION --cluster $CLUSTER --task <task ARN>
```

The cluster, task definition, execution role, log group and security group cost nothing while idle. A task running with a public IP does cost money, so do not leave one running.

A question worth sitting with

This lab runs the task in a public subnet with a wide-open security group, although the real backend belongs in a private subnet behind a load balancer. Why is it still reasonable to test in the simple setup first? Module 8 will change the networking, so the task does not run in a public subnet.